Solution: Count and Say

Problem Statement

Given an integer `n` greater than `0`, return the **n**th `count-and-say` sequence.

A `count-and-say` sequence is a string of digits created using specific rules given below.

- `countAndSay(1) = "1"`
- `countAndSay(2) = say countAndSay(1)`
- `countAndSay(n) = say countAndSay(n - 1)`

You start with "1", and for each subsequent string, you "read off" the digits of the previous string, counting the number of digits in groups of the same digit.

- For instance, `say "1223"` is read off as `one 1 + two 2 + one 3` or `112213`.

Examples

- **Example 1**:
  - **Input**: `n = 3`
  - **Expected Output**: `"21"`
  - **Justification**: The sequence is "1", "11" (one 1), "21" (two 1s).
- **Example 2**:
  - **Input**: `n = 4`
  - **Expected Output**: `"1211"`
  - **Justification**: Continuing from the last example, "21" is read as "one 2 + one 1" or "1211".
- **Example 3**:
  - **Input**: `n = 5`
  - **Expected Output**: `"111221"`
  - **Justification**: Continuing from the last example, "1211" is read as "one 1 + one 2 + two 1s" or "111221".

Solution

To solve this problem, we employ a recursive or iterative approach to build each string in the sequence from its predecessor. Initially, the sequence starts with "1". For each subsequent string, we iterate through the previous string, keeping track of the current digit and the count of its consecutive occurrences. When the current digit changes, we append its count and value to the new string. This process effectively 'counts and says' the digits of the previous string. This approach works because each string in the sequence is constructed by describing the composition of the previous string, ensuring the correctness and continuation of the sequence.

Step-by-step Algorithm

1. **Initialization**:
   - Define a string variable `result` and initialize it to `"1"`, representing the base case of the sequence. This variable will hold the current term in the sequence.
2. **Outer Loop - Sequence Construction**:
   - Iterate from `2` to `n` (inclusive) using a loop variable, say `i`. This loop is used to construct each term in the sequence, up to the `n`th term.
3. **Inner Loop - Term Construction**:
   - Inside the outer loop, initialize a new empty string `current` to build the `i`th term of the sequence.
   - Set two variables, `count` and `prev`. Initialize `count` to `1` (to count the occurrences of a digit) and `prev` to the first character of `result` (to track the current digit being counted).
4. **Counting and Saying**:
   - Iterate through each character of the `result` string using a loop variable, say `j`.
   - If the current character (`result[j]`) is the same as `prev`, increment `count`.
   - If it's different, append the value of `count` followed by `prev` to the `current` string. This is the "say" part. Reset `count` to `1` and update `prev` to the current character (`result[j]`).
5. **Finalizing the Current Term**:
   - After the inner loop, append the last counted `count` and `prev` values to the `current` string. This step is necessary to include the count of the last sequence of digits.
6. **Update for Next Iteration**:
   - Update the `result` string with the newly constructed `current` string. This makes `result` ready for the next iteration.
7. **Returning the Result**:
   - After the outer loop completes, return the `result` string, which now contains the `n`th term of the sequence.

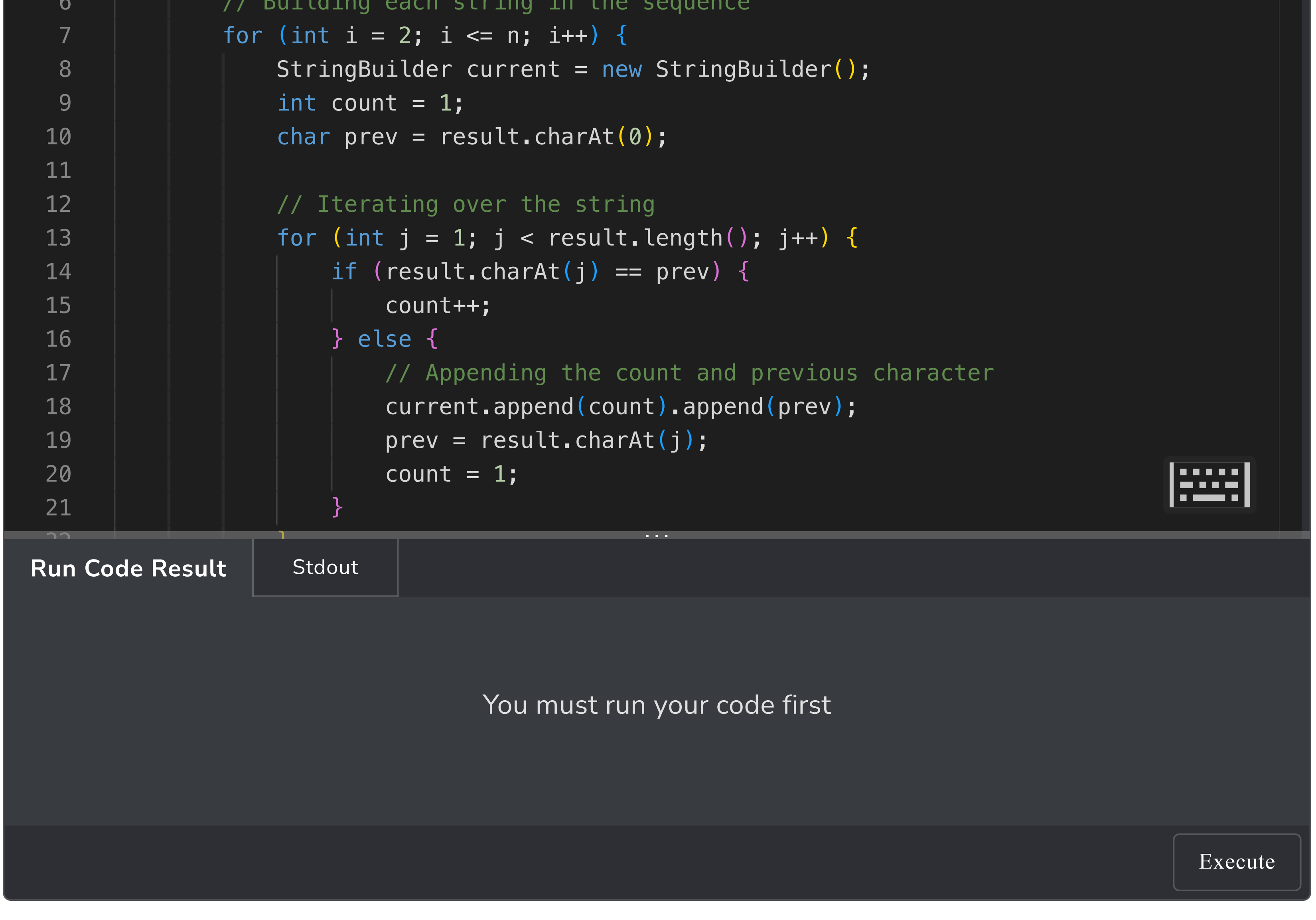
Algorithm Walkthrough

Let's consider the input: `5`.

1. **Initialization**
   - `result = "1"`. This is the base term of the sequence.
2. **1st Iteration** (**i = 2**)
   - Create `current` to build the next term.
   - Set `count = 1` (initial count of digits) and `prev = '1'` (the first character of `result`).
   - Iterate through `result`:
     - Only one character '1', so `current = "11"` (one 1).
3. **2nd Iteration** (**i = 3**)
   - Update `result = "11"`.
   - Reset `current`, `count = 1`, `prev = '1'`.
   - Iterate through `result`:
     - Two consecutive '1's, so `current = "21"` (two 1s).
4. **3rd Iteration** (**i = 4**)
   - Update `result = "21"`.
   - Reset `current`, `count = 1`, `prev = '2'`.
   - Iterate through `result`:
     - one '2', so `current = "12"`, then another '1', so `current = "1211"` (one 2, one 1).
5. **4th Iteration** (**i = 5**)
   - Update `result = "1211"`.
   - Reset `current`, `count = 1`, `prev = '1'`.
   - Iterate through `result`:
     - One '1', so `current = "11"`, then one '2', so `current = "1112"`, then two '1's, so `current = "111221"` (one 1, one 2, two 1s).
6. **End of Iterations**:
   - Update `result = "111221"`.
7. **Output**
   - The 5th term in the sequence is `"111221"`.

The final output for input `5` is `"111221"`, which is the 5th term in the "Count and Say" sequence.

Code



```java
public class Solution {
    public String countAndSay(int n) {
        // Base string for the sequence
        String result = "1";

        // Building each string in the sequence
        for (int i = 2; i <= n; i++) {
            StringBuilder current = new StringBuilder();
            int count = 1;
            char prev = result.charAt(0);

            // Iterating over the string
            for (int j = 1; j < result.length(); j++) {
                if (result.charAt(j) == prev) {
                    count++;
                } else {
                    // Appending the count and previous character
                    current.append(count).append(prev);
                    prev = result.charAt(j);
                    count = 1;
                }
```

You must run your code first

Complexity Analysis

Time Complexity

The time complexity is primarily determined by two factors: The **number of iterations** (up to `n`) and the **length of the sequence at each iteration** (which grows). The length of each sequence term increases at each step, and although it doesn't strictly double, it grows rapidly.

Therefore, the total time complexity can be approximated as `O(n * L(n))`, where `L(n)` is the length of the sequence at step `n`. This growth is super-linear but not strictly exponential.

Space Complexity

The space complexity is driven by the space needed to store the current and previous sequence strings. At any step, the space required is proportional to the length of the longest sequence, which is the sequence at step `n`. Therefore, the space complexity can be approximated as `O(L(n))`, where L(n) is the length of the sequence at step `n`.

← Previous
Count And Say (Medium)

Next →
Contiguous Array (Medium)

☐ Mark as Completed